

CS 224

LAB 3 – PRELIMINARY REPORT

Section No: 6

Name: Arda Akın

ID: 22402316

Date: 28 March 2026

Part B – Disassembly

Address	Machine Code (hex)	Assembly Format
0x00	0x20020005	addi \$v0, \$zero, 5
0x04	0x2003000c	addi \$v1, \$zero, 12
0x08	0x2067fff7	addi \$a3, \$v1, -9
0x0c	0x00e22025	or \$a0, \$a3, \$v0
0x10	0x00642824	and \$a1, \$v1, \$a0
0x14	0x00a42820	add \$a1, \$a1, \$a0
0x18	0x10a7000a	beq \$a1, \$a3, 10
0x1c	0x0064202a	slt \$a0, \$v1, \$a0
0x20	0x10800001	beq \$a0, \$zero, 1
0x24	0x20050000	addi \$a1, \$zero, 0
0x28	0x00e2202a	slt \$a0, \$a3, \$v0
0x2c	0x00853820	add \$a3, \$a0, \$a1
0x30	0x00e23822	sub \$a3, \$a3, \$v0
0x34	0xac670044	sw \$a3, 68(\$v1)
0x38	0x8c020050	lw \$v0, 80(\$zero)
0x3c	0x08000011	j 0x44
0x40	0x20020001	addi \$v0, \$zero, 1
0x44	0xac020054	sw \$v0, 84(\$zero)
0x48	0x08000012	j 0x48

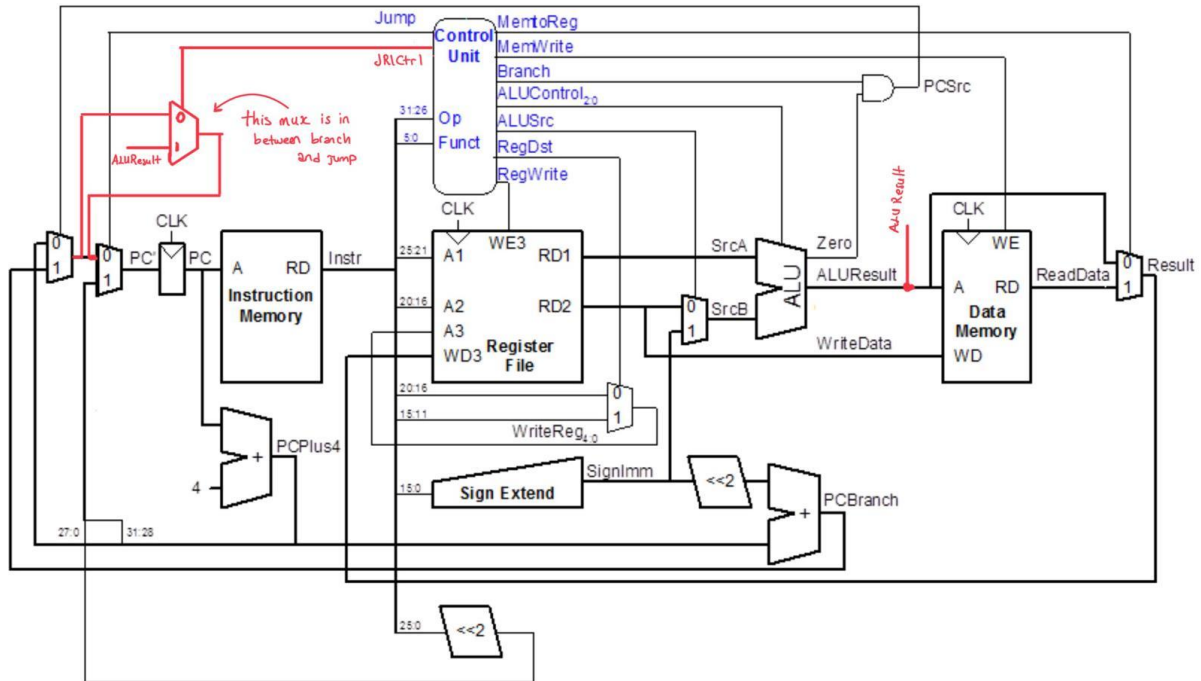
Part C – RTL Expressions

- jri – jump register immediate
IM[PC]
 $PC \leftarrow R[rs] + \text{SignExtImm}$
- slti – set less than immediate
IM[PC]
 $R[rt] \leftarrow (R[rs] < \text{SignExtImm}) ? 1 : 0$
 $PC \leftarrow PC + 4$

Part D – Datapath Modifications

No changes needed for slti

A new multiplexer added for jri, to directly put the ALU result to PC. New control signal is driven from the control unit.



Part E – Updated Control Tables

Table 1: Main Decoder for Original10

Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemToReg	ALUOp	Jump	JR/ICtrl
R-type	000000	1	1	0	0	0	0	10	0	0
lw	100011	1	0	1	0	0	1	00	0	0
sw	101011	0	X	1	0	1	X	00	0	0
beq	000100	0	X	0	1	0	X	01	0	0
addi	001000	1	0	1	0	0	0	00	0	0
j	000010	0	X	X	X	0	X	XX	1	0
jri	111010	0	X	1	0	0	X	00	0	1
slti	001010	1	0	1	0	0	0	11	0	0

Table 2: ALU Decoder for Original10

ALUOp	Funct	ALUControl
00	X	010
01	X	110
1X	100000 (add)	010
1X	100010 (sub)	110
1X	100100 (and)	000
1X	100101 (or)	001
1X	101010 (slt)	111
11	X	111

Part F – Test Program in MIPS

addi \$t0, \$zero, 10 # t0 = 10

addi \$t1, \$zero, 20 # t1 = 20

addi \$t2, \$zero, -5 # t2 = -5

#R-type tests

add \$t3, \$t0, \$t1 # t3 = 30

sub \$t4, \$t1, \$t0 # t4 = 10

and \$t5, \$t0, \$t1 # t5 = 0

or \$t6, \$t0, \$t1 # t6 = 30

slt \$t7, \$t0, \$t1 # t7 = 1 (10 < 20)

#I-type Tests

slti \$s0, \$t2, -3 # -5 < -3 → 1

slti \$s1, \$t0, 5 # 10 < 5 → 0

#Memory tests

addi \$a0, \$zero, 100 # base address = 100

sw \$t3, 0(\$a0) # mem[100] = 30

lw \$s2, 0(\$a0) # s2 = 30

#Branch Tests

beq \$t0, \$t0, LABEL1 # always taken

addi \$s3, \$zero, 999 # must be skipped

LABEL1: addi \$s3, \$zero, 111 # s3 = 111

Jump test

j LABEL2

addi \$s4, \$zero, 999 # must be skipped

LABEL2: addi \$s4, \$zero, 222 # s4 = 222

#jri test

addi \$a1, \$zero, 200 # base = 200 jri 8(\$a1) # jump to 208

addi \$s5, \$zero, 999 # must be skipped

addi \$s5, \$zero, 333 # s5 = 333

END: beq \$zero, \$zero, END

(Since there is no syscall defined in Original10, I did not use syscall to print and see results)

Part G – Signal Values for sw \$a1, 8(\$a0)

- ALU Input A: 0x00000010 (\$a0 = 0x10)
- ALU Input B: 0x00000008 (Sign immediate value)
- ALU Result: 0x00000018 (Addition)
- WriteData: 0x000000FF (\$a1 = 0xFF)
- RegWrite: 0 (sw does not write to the register file)

Part F – System Verilog Code

- Maindec: maindec should be changed, we need to add new opcodes for jri and slti. We also need to add JRICtrl output.

Modified System Verilog Code:

```
module maindec (input logic [5:0] op,
                output logic memtoreg, memwrite, branch,
                output logic alusrc, regdst, regwrite, jump,
                output logic jrictrl, // NEW: jri control
                output logic [1:0] aluop);

    logic [9:0] controls; // 1 extra bit for jrictrl

    assign {regwrite, regdst, alusrc, branch, memwrite, memtoreg, aluop, jump,
    jrictrl} = controls;

    always_comb
    case (op)
        6'b000000: controls = 10'b1100001000; // R-type
        6'b100011: controls = 10'b1010010000; // lw
        6'b101011: controls = 10'b0010100000; // sw
        6'b000100: controls = 10'b0001000100; // beq
        6'b001000: controls = 10'b1010000000; // addi
        6'b000010: controls = 10'b0000000010; // j
        6'b111010: controls = 10'b0010000001; // jri (op=0x3A) RegWrite=0,
        RegDst=X, ALUSrc=1, Branch=0 MemWrite=0, MemToReg=X, ALUOp=00(add) Jump=0, JRICtrl=1
        6'b001010: controls = 10'b1010001100; // slti (op=0x0A) RegWrite=1,
        RegDst=0, ALUSrc=1, Branch=0 MemWrite=0, MemToReg=0, ALUOp=11(slt) Jump=0, JRICtrl=0
        default: controls = 10'bxxxxxxxxxx;
    endcase
endmodule
```

- Aludec: aludec should be changed, we need to add aluop=11 case for slti.

Modified System Verilog Code:

```
module aludec (input logic [5:0] funct,
               input logic [1:0] aluop,
```

```

        output logic [2:0] alucontrol);

always_comb
case (aluop)
    2'b00: alucontrol = 3'b010; // add (lw/sw/addi/jri)
    2'b01: alucontrol = 3'b110; // sub (beq)
    2'b11: alucontrol = 3'b111; // slti
    default: case (funct) // R-TYPE
        6'b100000: alucontrol = 3'b010; // add
        6'b100010: alucontrol = 3'b110; // sub
        6'b100100: alucontrol = 3'b000; // and
        6'b100101: alucontrol = 3'b001; // or
        6'b101010: alucontrol = 3'b111; // slt
        default: alucontrol = 3'bxxx;
    endcase
endcase

Endmodule

```

- Controller: controller must be updated. JRICtrl is added and pcsrc logic updated for jri.

Modified System Verilog Code:

```

module controller (input logic [5:0] op, funct,
    input logic zero, output logic memtoreg, memwrite,
    output logic pcsrc, alusrc,
    output logic regdst, regwrite, output logic jump,
    output logic jrictrl, // NEW output logic [2:0] alucontrol);

    logic [1:0] aluop;

    logic branch;

    maindec md (op, memtoreg, memwrite, branch, alusrc, regdst, regwrite, jump, jrictrl,
    aluop); // jrictrl added

    aludec ad (funct, aluop, alucontrol);

    assign pcsrc = branch & zero;

```

Endmodule

- Datapath: datapath must be modified. We need to add mux for jri PC selection.

Modified System Verilog Code:

```
module datapath (input logic clk, reset,
                 input logic memtoreg, pcsrc,
                 input logic alusrc, regdst,
                 input logic regwrite, jump,
                 input logic jrictrl, // NEW
                 input logic [2:0] alucontrol,
                 output logic zero,
                 output logic [31:0] pc,
                 input logic [31:0] instr,
                 output logic [31:0] aluout, writedata,
                 input logic [31:0] readdata);

    logic [4:0] writereg;

    logic [31:0] pcnext, pcnextbr, pcplus4, pcbranch;

    logic [31:0] signimm, signimmsh, srca, srcb, result;

    logic [31:0] pcnextjri; // new: PC after jri mux

    // next PC logic

    flopr #(32) pcreg (clk, reset, pcnext, pc);

    adder pcadd1 (pc, 32'b100, pcplus4);

    sl2 immsh (signimm, signimmsh);

    adder pcadd2 (pcplus4, signimmsh, pcbranch);

    // Branch mux

    mux2 #(32) pcbrmux (pcplus4, pcbranch, pcsrc, pcnextbr);

    // jri mux — select ALU result as PC if jrictrl=1

    mux2 #(32) pcjrimux (pcnextbr, aluout, jrictrl, pcnextjri);

    // Jump mux

    mux2 #(32) pcmux (pcnextjri, {pcplus4[31:28], instr[25:0], 2'b00}, jump, pcnext);

    // register file logic

    regfile rf (clk, regwrite, instr[25:21], instr[20:16], writereg, result, srca, writedata);
```

```

mux2 #(5) wrmux (instr[20:16], instr[15:11], regdst, writereg);
mux2 #(32) resmux (aluout, readdata, memtoreg, result);
signext se (instr[15:0], signimm);
// ALU logic
mux2 #(32) srcbmux (writedata, signimm, alusrc, srcb);
alu alu (srca, srcb, alucontrol, aluout, zero);
Endmodule

```

- MIPS (top level): Top level mips must be updated. We need to update port list for jrictrl internal wire.

Modified System Verilog Code:

```

module mips (input logic clk, reset,
             output logic [31:0] pc,
             input logic [31:0] instr,
             output logic memwrite,
             output logic [31:0] aluout, writedata,
             input logic [31:0] readdata);

logic memtoreg, pcsrc, zero, alusrc, regdst, regwrite, jump;
logic jrictrl; // new
logic [2:0] alucontrol;

controller c (instr[31:26], instr[5:0], zero, memtoreg, memwrite, pcsrc, alusrc, regdst,
             regwrite, jump, jrictrl, alucontrol);

datapath dp (clk, reset, memtoreg, pcsrc, alusrc, regdst, regwrite, jump, jrictrl,
            alucontrol, zero, pc, instr, aluout, writedata, readdata);

endmodule

```